# iOS Merchant Authorization & Accounts Guide

ON THIS PAGE

Prerequisites

Retrieving Merchant Authorization Token

Retrieving Merchant Accounts

The **iOS Merchant Authorization & Accounts Guide** explains how to securely retrieve and manage merchant authorization tokens and account details for use with the iOS SDK. This guide will cover the following areas:

- **Authorization Token Retrieval**: Steps to generate and request a bearer token using client credentials, including handling token expiration and renewal.
- **Merchant Account Retrieval**: How to retrieve merchant account details using the access token, including identifying accounts eligible for mobile transactions.

## Prerequisites

Once you've successfully completed Deluxe Merchant onboarding and received your API keys, you're ready to validate your client application's credentials and retrieve an authorization token.

Before beginning the integration, ensure you have the following:

- **A Deluxe Merchant Account:** You will need an active Merchant Account.
- **Access Token & Client Application Credentials:** To integrate with the Deluxe Mobile SDK, you'll need an Access Token and Client App Credentials for authentication and configuration of your application. Our Integration Team securely provides these credentials.
- **Access to Deluxe Developer Portal:** The portal provides additional resources, API documentation, and integration tools.

Note: *If you don't have a Deluxe Merchant Account or Access Tokens and Secret Key, simply visit Merchant Services Get Started and contact us today.*

```
let merchantAuthService =
MerchantAuthorizationService ()
```

### STEP 2: Create a GetMerchantAuthorizationTokenRequest object

You need to create an apiKey using clientId and clientSecret using the below format and encode it using Base64

{clientId:clientSecret}

```
let apiKey = "\(clientId):\(clientSecret)"
guard let apiData = apiKey.data(using: .utf8) else
{ return }
let encodedAPIKey = apiData.base64EncodedString()

let getAuthTokenRequest =
GetMerchantAuthorizationTokenRequest
(apiKey:encodedAPIKey)
```

### STEP 3: Retrieve the authorization token using getMerchantAuthorizationToken()

Invoke getMerchantAuthorizationToken() method by passing the GetMerchantAuthorizatinTokenRequest object to retrieve the authorization token. This method will return the GetMerchantAuthorizationTokenResponse object on success scenarios and error if any error occurred in the execution.

```
merchantAuthService.getMerchantAuthorizationToken(r
equest: request) { result, error in  }
```

### Step 4: Retrieve the authorization token from Response

Upon successful execution GetMerchantAuthorizationTokenResponse object will contain the MerchantAuthorizationToken object which contains the bearer token and its expiry in seconds. This bearer token can be used as an input to the most of the SDK methods.

```
merchantAuthService.getMerchantAuthorizationToken(r
equest: request) { result, error in
guard let bearerToken =
result?.response?.bearerToken else {
return
}
```

### Expiry of MerchantAuthorizationToken

The received Bearer Token will get expired in the given expiry period in seconds, which can be seen in the expiresIn property of MerchantAuthorization object. Currently the expiry is set to 86400 seconds.

If the bearer token is expired , then all your subsequent framework calls will result into errors , and which will result in unAuthorizedAccess (401).

Client application should use the same getMerchantAuthorizationToken() above to regain the new bearer token once the token is expired.

## Retrieving Merchant Accounts

You can retrieve all merchant accounts using the MerchantAccountService class by calling the getMerchantAccounts() method. This returns a list of accounts associated with the authenticated user. Each account includes a hasMobile flag within its Mids object, indicating whether it supports mobile transactions. Depending on your onboarding configuration, one or more accounts may be mobile-enabled. If none of the Mids have hasMobile set to true, mobile features will not be available. Each account also includes key properties such as mid, tid, dba, and accessToken, which are required for most SDK operations.

### STEP 1: Create a MerchantAccountService object

```
let merchantAccountService =
MerchantAccountService()
```

### STEP 2: Create a MerchantAccountRequest object

```
let getMerchantAccountRequest =
GetMerchantAccountsRequest(userId: nil,
accessToken: accessToken)
```

- **userID** will always be nil for the current version of the SDK.

### STEP 3: Retrieve Merchant Account Details Using getMerchantAccounts()

The MerchantAccountService class is included in the library. To retrieve merchant account details, simply call the getMerchantAccounts() method.

```
let merchantAccountService =
MerchantAccountService()
let response =
merchantAccountService.getMerchantAccounts(request:
requestParameter, bearerToken: bearerToken)
```

Parameter bearer token should be the bearertoken you received as part of the the previous section. Please Refer to the **Retrieving Merchant Authorization Token** section above.

### Step 4: Handling the GetMerchantAccountResponse from getMerchantAccounts()

When you call the getMerchantAccounts() method, you'll receive a MerchantAccountResponse containing all merchant accounts linked to the authenticated user. Each account includes a hasMobile flag within its Mids object, which indicates whether that account supports mobile transactions. If none of the Mids have hasMobile = true, mobile features will not be available for that merchant. This check helps ensure you're selecting an account that’s properly configured for mobile payments.

Note: *Please refer to the Complete SDK Reference for additional details iOS SDK Reference Guide*

deluxe. connect

Payments API

Deluxe.com | Privacy Policy | Do Not Sell My Personal Information

Deluxe Corporation, 801 S Marquette Ave,
Minneapolis, MN 55402.

© 2026 Deluxe Corporation